Single-Cell RNA-seq Analysis Pipeline

From Raw Reads to Differential Expression and GO Enrichment

2026-03-31

Contents

1 Overview **3**
1.1 Pipeline overview . . . 3

2 Requirements **4**
2.1 Hardware . . . 4
2.2 Docker installation . . . 4
2.2.1 Linux (Ubuntu/Debian) . . . 4
2.2.2 macOS / Windows . . . 4
2.3 Pulling the container . . . 4
2.4 Starting an interactive session . . . 4

3 Step 1: Reference Genome Preparation **5**
3.1 Obtaining the genome and annotation . . . 5
3.2 Filtering to protein-coding genes (recommended) . . . 6
3.3 Building the CellRanger reference index . . . 6

4 Step 2: Read Quantification (CellRanger) **7**
4.1 FASTQ file naming convention . . . 7
4.2 Running cellranger count . . . 7

5 Step 3: Ambient RNA Removal (CellBender) — Optional **8**

6 Step 4: Pipeline Configuration **9**
6.1 Output directory structure . . . 10

7 Step 5: Quality Control **12**
7.1 Pre-filter assessment . . . 12
7.2 Filtering and doublet detection . . . 13

8 Step 6: Data Merging, Normalisation, and Preprocessing **14**

9 Step 7: Batch Correction (Harmony) and Dimension Selection **15**
9.1 Elbow plot . . . 15

10 Step 8: Clustering Resolution Selection (Clustree) **16**

11 Step 9: Final Clustering **17**

12 Step 10: Marker Gene Identification and Cell-Type Annotation **18**
12.1 Marker gene dot plot (pre-annotation guide) 18
12.2 Literature-based annotation 18
12.3 Reference-based annotation 18
12.4 Annotated clustree 19

13 Step 11: Gene Expression Visualisation **20**

14 Step 12: Cell-Type Grouping and Manual Curation **21**
14.1 Grouping fine annotations 21
14.2 Manual subclustering and curation 21

15 Step 13: Export to Python (h5ad) **24**

16 Step 14: Global Pseudobulk Replicate Correlation **26**

17 Step 15: Pseudobulk per Cell Type and DESeq2 Differential Expression **27**
17.1 Pseudobulk aggregation 27
17.2 DESeq2 analysis 27
17.3 Volcano plots and differential heatmaps 28

18 Step 16: GO Enrichment Analysis **30**

19 Adapting the Pipeline to Other Organisms **32**

20 Output File Summary **33**

1 Overview

This document describes a complete, reproducible computational pipeline for analysing single-cell RNA sequencing (scRNA-seq) data generated on the 10x Genomics Chromium platform. The workflow starts from raw FASTQ files, aligns reads to a reference genome, removes ambient RNA contamination, performs quality control, integrates multiple samples by correcting batch effects, clusters cells, assigns cell-type identities, and finally performs pseudobulk differential expression and gene ontology (GO) enrichment analysis.

All software runs inside a Docker container (`mvergara19/scrnaseq_docker:latest`), which packages R 4.5, Python 3, Seurat 5, harmony, DESeq2, CellRanger 9.0.1, CellBender, and all dependencies into a single reproducible environment. No local R or Python installation is needed beyond Docker itself.

The pipeline is **organism-agnostic**. The examples throughout this document use *Arabidopsis thaliana* under three nitrogen conditions (0N, 0.5N, and 5N), but adapting it to human, mouse, or any other species requires changing only three parameters: the mitochondrial gene prefix, the GO annotation database, and the key type.

The pipeline is divided into two chapters:

- **Chapter 1** covers the complete single-cell analysis: from raw data loading through quality control, integration, clustering, annotation, and export to Python.
- **Chapter 2** covers the downstream bulk-level analyses: pseudobulk differential expression with DESeq2, volcano plots, heatmaps, and GO enrichment.

1.1 Pipeline overview

The figure below summarises the complete workflow. It is generated automatically by `scrnaseq_pipeline.R` (Section 0) and saved to `results/00_workflow/pipeline_workflow.pdf`.

---

2 Requirements

2.1 Hardware

Resource	Minimum	Recommended
RAM	16 GB	64 GB (datasets $>$ 30,000 cells)
Storage	50 GB free	200 GB
CPU	4 cores	24 cores (for CellRanger)

2.2 Docker installation

2.2.1 Linux (Ubuntu/Debian)

```
sudo apt-get update && sudo apt-get install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER   # log out and back in after this
```

2.2.2 macOS / Windows

Download Docker Desktop from `https://www.docker.com/products/docker-desktop/` and follow the installer. On Windows, WSL 2 is required and the installer guides the setup.

2.3 Pulling the container

```
docker pull mvergara19/scrnaseq_docker:latest
```

2.4 Starting an interactive session

```
docker run --rm -it -v $(pwd):/workspace mvergara19/scrnaseq_docker:latest bash
```

The `-v` flag mounts the current directory into `/workspace` inside the container. All input data and output files should live under this directory.

---

3 Step 1: Reference Genome Preparation

Before quantifying gene expression, a species-specific reference must be built from two files:

- **Genome sequence** (`.fa` or `.fasta`) — the full DNA sequence of the organism, chromosome by chromosome.
- **Gene annotation** (`.gtf`) — a table that maps each gene to its coordinates on the genome (chromosome, start, end, strand) and defines exon boundaries for transcript assembly.

These two files are combined by CellRanger to build an indexed reference, which is used to align sequencing reads to the correct gene loci.

This step is performed once per organism. Once the reference is built it can be reused for all samples from the same species and annotation version.

3.1 Obtaining the genome and annotation

Reference files are freely available from public databases. Choose the source that matches your organism:

Organism	Recommended source
Arabidopsis thaliana	Ensembl Plants (`plants.ensembl.org`)
Human	10x Genomics pre-built references (fastest option) or Ensembl (`ensembl.org`)
Mouse	10x Genomics pre-built references or Ensembl
Other plants / model organisms	Ensembl Plants, Phytozome, or organism-specific databases

Human and mouse — 10x Genomics provides ready-to-use indexed references that can be downloaded directly and require no further building:

```
# GRCh38 (human)
curl -O https://cf.10xgenomics.com/supp/cell-exp/refdata-gex-GRCh38-2024-A.tar.gz

# mm10 (mouse)
curl -O https://cf.10xgenomics.com/supp/cell-exp/refdata-gex-mm10-2020-A.tar.gz
```

Other organisms — download the genome FASTA and GTF from Ensembl (or the relevant database) and build the reference with `cellranger mkref` (see below).

For *Arabidopsis thaliana* (Ensembl Plants release 53):

```
wget http://ftp.ensemblgenomes.org/pub/plants/release-53/fasta/arabidopsis_thaliana/dna/Arabido

wget http://ftp.ebi.ac.uk/ensemblgenomes/pub/release-53/plants/gtf/arabidopsis_thaliana/Arabido

gzip -d Arabidopsis_thaliana.*
```

3.2 Filtering to protein-coding genes (recommended)

CellRanger recommends restricting the annotation to protein-coding genes to reduce multi-mapping reads from non-coding or pseudo-genes and improve quantification accuracy. The `cellranger mkgtf` command adds a simple attribute filter:

```
cellranger mkgtf \
    Arabidopsis_thaliana.TAIR10.53.gtf \
    Arabidopsis_thaliana.TAIR10.53.filtrado.gtf \
    --attribute=gene_biotype:protein_coding
```

For human/mouse GTFs the equivalent flag is `--attribute=gene_type:protein_coding`.

3.3 Building the CellRanger reference index

```
cellranger mkref \
    --genome=Arabidopsis_thaliana.TAIR10.53.transcriptome \
    --fasta=Arabidopsis_thaliana.TAIR10.dna.toplevel.fa \
    --genes=Arabidopsis_thaliana.TAIR10.53.filtrado.gtf
```

This generates an indexed reference directory used by all subsequent `cellranger count` calls. Building the reference typically takes 20–40 minutes and requires ~8 GB RAM.

---

4 Step 2: Read Quantification (CellRanger)

CellRanger aligns cDNA reads to the reference genome using STAR, corrects cell barcodes, collapses UMIs, and produces a filtered count matrix in which each column is a cell and each row is a gene.

4.1 FASTQ file naming convention

Each 10x Genomics library produces three FASTQ files per sequencing lane:

Suffix	Content
`*_I1_001.fastq`	Sample index (8 bp)
`*_R1_001.fastq`	Cell barcode (16 bp) + UMI (12 bp)
`*_R2_001.fastq`	cDNA read (the actual transcript sequence)

4.2 Running cellranger count

```
cellranger count \
    --localcores=24 \
    --id=run_count \
    --fastqs=/workspace/fastqs \
    --sample=SRR8485805 \
    --transcriptome=/workspace/Arabidopsis_thaliana.TAIR10.53.transcriptome
```

Replace `--sample` with the filename prefix of your FASTQ files, and `--transcriptome` with the path to the reference built in Step 1. This step takes several hours per sample. The key output is `run_count/outs/raw_feature_bc_matrix.h5` (required by CellBender) and `run_count/outs/filtered_feature_bc_matrix/` (used when skipping CellBender).

---

5 Step 3: Ambient RNA Removal (CellBender) — Optional

> **This step is optional.** If you choose to skip it, set `USE_CELLBENDER <- FALSE` in Step 4 and point `samples$file` to the CellRanger `filtered_feature_bc_matrix/` directory instead of the CellBender HDF5 output.

During library preparation, lysed cells release mRNA into the cell suspension. This ambient RNA is captured by empty droplets and inflates the counts of certain genes across all cells — potentially misleading cell-type annotation and differential expression. CellBender uses a deep generative model to estimate the ambient RNA profile and subtract it from each cell's count vector.

When to use CellBender:

- Datasets with visibly high ambient RNA (many cells expressing housekeeping genes at unexpectedly similar levels)
- Any experiment where clean cell-type separation is critical
- Recommended for all high-quality analyses

When you can skip it:

- Quick exploratory analyses
- Datasets with very low ambient RNA (confirmed by low empty-droplet counts)
- Situations where compute time is a constraint

```
cellbender remove-background \
    --input  /workspace/raw_feature_bc_matrix.h5 \
    --output /workspace/Sample_cellbender_output.h5
```

The corrected output (`Sample_cellbender_output_filtered.h5`) is the recommended input for all downstream R analyses. Running CellBender on a GPU significantly reduces computation time; to enable GPU support, launch the container with `--gpus all` and ensure the NVIDIA Container Toolkit is installed on the host.

---

6 Step 4: Pipeline Configuration

All experiment-specific parameters are defined at the top of `scrnaseq_pipeline.R`. Edit only this block before running the pipeline.

Three helper R scripts are sourced at startup. Each is **fully documented** (function signatures, parameters, return values, and examples) in the repository at `https://github.com/cliford2001/ScRNASeq-Docker`:

File	Purpose
`load_libraries.R`	Loads all required R packages
`ScRNA_Analysis_Functions.R`	Core analysis functions (QC, clustering, annotation, DE, GO)
`custom_seurat.R`	Custom Seurat plot utilities (cluster bar charts)

```
# Directory containing the pipeline helper scripts
PIPELINE_DIR <- "/workspace/ScRNASeq-Docker"

# Root directory for your data and results
DATA_DIR <- "/workspace/ScRNA"
base_dir <- file.path(DATA_DIR, "results")   # all outputs go here

# Input format:
#   TRUE  → load CellBender-filtered HDF5 files (recommended)
#   FALSE → load CellRanger filtered_feature_bc_matrix/ directly
USE_CELLBENDER <- TRUE

# Load libraries and helper functions
# (full documentation at https://github.com/cliford2001/ScRNASeq-Docker)
source(file.path(PIPELINE_DIR, "load_libraries.R"))
source(file.path(PIPELINE_DIR, "custom_seurat.R"))
source(file.path(PIPELINE_DIR, "ScRNA_Analysis_Functions.R"))

set.seed(1807)

# Sample manifest
samples <- list(
  list(file = "cellbender/Sample_0N_cellbender_filtered.h5",
       label = "0N",      condition = "0N"),
  list(file = "cellbender/Sample_05N_R1_cellbender_filtered.h5",
       label = "0.5N_R1", condition = "0.5N"),
  list(file = "cellbender/Sample_5N_R1_cellbender_filtered.h5",
       label = "5N_R1",   condition = "5N")
)

# Plot colors
colors <- c("0N" = "#66c2a5", "0.5N_R1" = "#fc8d62", "5N_R1" = "#8da0cb")
```

6.1 Output directory structure

Results are automatically organised into numbered subdirectories so every output file has a clear location:

```
results/
  00_workflow/    Pipeline overview figure
  01_qc/          Pre-filter QC violin plots
  02_filtering/   Post-filter QC violin plots
  03_integration/ Pre/post-Harmony UMAPs
  04_clustering/  Elbow plot, clustree, final cluster UMAPs, cell count
  05_annotation/  Dotplot, marker tables, annotated UMAPs, annotated clustree
  06_expression/  Violin and feature plots
  07_curation/    Grouped and curated UMAPs, subclustering inspection figure
  08_export/      h5ad files for Python
  09_pseudobulk/  Correlation heatmap, pseudobulk count matrices
  10_deseq2/      DESeq2 result CSV files
  11_volcano/     Volcano plot PDFs
  12_heatmaps/    Log2FC heatmap PDFs
  13_go/          GO enrichment balloon plots and tables
```

CHAPTER 1 — Single-Cell Analysis

7 Step 5: Quality Control

Quality control (QC) identifies and removes low-quality cells before downstream analysis. Three metrics are computed per cell:

- **nFeature_RNA** — number of distinct genes detected. Very low values indicate empty droplets; unusually high values may indicate doublets.
- **nCount_RNA** — total UMI count; correlated with sequencing depth.
- **percent.mt / percent.cp** — percentage of reads mapping to mitochondrial (or chloroplast, for plants) genes. Elevated values indicate damaged cells that have lost cytoplasmic mRNA.

7.1 Pre-filter assessment

Samples are loaded and mitochondrial/chloroplast genes are annotated. Violin plots are generated before any filtering to guide threshold selection.

```
# Change mt_pattern and cp_pattern for your organism:
#   Arabidopsis : mt_pattern = "^ATMG"  |  cp_pattern = "^ATCG"
#   Human       : mt_pattern = "^MT-"   |  cp_pattern = NULL
#   Mouse       : mt_pattern = "^mt-"   |  cp_pattern = NULL
mt_pattern <- "^ATMG"
cp_pattern <- "^ATCG"

# USE_CELLBENDER controls how samples are loaded (set in Step 4 config)
if (USE_CELLBENDER) {
  seurat_list_raw <- lapply(samples, load_sample,
                            mt_pattern = mt_pattern,
                            cp_pattern = cp_pattern)
} else {
  # Load directly from CellRanger filtered_feature_bc_matrix/ directories
  seurat_list_raw <- lapply(samples, function(s) {
    mat <- Read10X(data.dir = file.path(DATA_DIR, s$file))
    obj <- CreateSeuratObject(counts = mat, project = s$label,
                              min.cells = 3, min.features = 200)
    obj$condition       <- s$condition
    obj[["percent.mt"]] <- PercentageFeatureSet(obj, pattern = mt_pattern)
    if (!is.null(cp_pattern))
      obj[["percent.cp"]] <- PercentageFeatureSet(obj, pattern = cp_pattern)
    obj
  })
}
names(seurat_list_raw) <- sapply(samples, `[[`, "label")

output_dir <- dir_01   # save to 01_qc/
plots_pre  <- imap(seurat_list_raw, ~ plot_qc_violin_grid(.x, .y, colors[[.y]]))
save_qc(plots_pre, "qc_prefilter.pdf")
```

7.2 Filtering and doublet detection

Based on the pre-filter distributions, thresholds are applied. DoubletFinder is then run per sample to identify and remove co-encapsulated cell pairs.

```
# Adjust thresholds after inspecting 01_qc/qc_prefilter.pdf
min_features <- 200   # minimum detected genes per cell
max_mt       <- 5     # maximum mitochondrial read percentage

output_dir <- dir_02  # save to 02_filtering/
seurat_list <- lapply(seurat_list_raw, filter_sample,
                      min_features = min_features, max_mt = max_mt)
names(seurat_list) <- sapply(samples, `[[`, "label")

plots_post <- imap(seurat_list, ~ plot_qc_violin_grid(.x, .y, colors[[.y]]))
save_qc(plots_post, "qc_postfilter.pdf")
```

---

8 Step 6: Data Merging, Normalisation, and Preprocessing

After quality control, all samples are merged into a single Seurat object and preprocessed through normalisation, variable feature selection, scaling, PCA, and UMAP.

```
output_dir <- dir_03   # save to 03_integration/

pbmc_harmony <- reduce(seurat_list, merge) %>%
  NormalizeData(verbose = FALSE) %>%
  FindVariableFeatures(nfeatures = 2000, verbose = FALSE) %>%
  ScaleData(verbose = FALSE) %>%
  RunPCA(npcs = 30, verbose = FALSE) %>%
  RunUMAP(reduction = "pca", dims = 1:30, verbose = FALSE)

pbmc_harmony$orig.ident_uni <- pbmc_harmony$condition

save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
         "umap_preharmony.pdf")
```

The UMAP below shows all cells before batch correction, coloured by sample. Separation of cells by sample confirms the presence of batch effects that must be corrected.

---

9 Step 7: Batch Correction (Harmony) and Dimension Selection

Harmony iteratively adjusts the PCA embedding to remove sample-specific shifts while preserving biological signal.

```
# dims_use : number of Harmony dimensions to use downstream (default 1:30)
# k_param  : number of nearest neighbors for the cell graph (default 30)
dims_use <- 1:30
k_param  <- 30

pbmc_harmony <- pbmc_harmony %>%
  RunHarmony("orig.ident", plot_convergence = FALSE)
```

9.1 Elbow plot

The number of informative PCA dimensions is assessed using a k-means WSS elbow plot. The optimal value corresponds to the inflection point of the curve.

```
output_dir <- dir_04   # save to 04_clustering/

k_range  <- 2:40
pca_data <- Embeddings(pbmc_harmony, "pca")[, dims_use]
wss      <- sapply(k_range, function(k)
  kmeans(pca_data, centers = k, nstart = 10)$tot.withinss)

elbow_plot <- ggplot(data.frame(k = k_range, wss = wss), aes(k, wss)) +
  geom_line() + geom_point() +
  labs(x = "Number of clusters (k)", y = "Within-cluster sum of squares") +
  theme_minimal()

save_pdf(elbow_plot, "elbow_plot.pdf", w = 8, h = 6)
```

---

10 Step 8: Clustering Resolution Selection (Clustree)

Clustering is run across a range of resolutions and visualised as a tree to select a stable partition.

```
# resolutions_test : range of resolutions to evaluate
# Inspect 04_clustering/clustree.pdf and elbow_plot.pdf before setting
# cluster_resolution in Step 9
resolutions_test <- c(0.15, 0.30, 0.50, 0.8, 1.0)

output_dir <- dir_04   # save to 04_clustering/

clu <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = dims_use, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = dims_use,
                k.param = k_param, verbose = FALSE)

for (res in resolutions_test)
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)

save_pdf(clustree(clu, prefix = "RNA_snn_res."), "clustree.pdf", w = 14, h = 14)
```

---

11 Step 9: Final Clustering

The chosen resolution is applied for the final clustering and results are visualised on the Harmony-corrected UMAP. A bar chart of cells per sample is also produced here, immediately after integration quality is confirmed.

```
# cluster_resolution : set this after inspecting elbow_plot.pdf and clustree.pdf
cluster_resolution <- 0.3

output_dir <- dir_04   # save to 04_clustering/

pbmc_harmony <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = dims_use, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = dims_use,
                k.param = k_param, verbose = FALSE) %>%
  FindClusters(resolution = cluster_resolution, algorithm = 4, verbose = FALSE)

# UMAP coloured by sample identity (confirms successful batch correction)
save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
         "umap_postharmony.pdf")

# UMAP coloured by Seurat cluster ID
save_pdf(DimPlot(pbmc_harmony, group.by = "seurat_clusters",
                 cols = sample(colors(distinct = TRUE),
                               length(unique(pbmc_harmony$seurat_clusters)))),
         "umap_seuratclusters.pdf")

# Cell count per sample
cell_count_plot <- ggplot(pbmc_harmony@meta.data,
                          aes(x = orig.ident, fill = orig.ident)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = after_stat(count)),
            vjust = -0.4, size = 4) +
  scale_fill_manual(values = colors) +
  theme_bw(base_size = 14) +
  labs(title = "Total cells per sample after filtering and integration",
       x = "Sample", y = "Cell count") +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 45, hjust = 1))

save_pdf(cell_count_plot, "cell_count_per_sample.pdf", w = 8, h = 6)
```

---

12 Step 10: Marker Gene Identification and Cell-Type Annotation

12.1 Marker gene dot plot (pre-annotation guide)

Before assigning formal cell-type labels, a dot plot shows how the bibliography marker genes are distributed across the numbered Seurat clusters. Clusters that strongly express a known marker (e.g., AT5G26000 for Guard Cell) can then be labelled accordingly in the annotation step below. Dot size encodes the fraction of expressing cells; colour encodes mean scaled expression.

The `biblio_marks.txt` file is a tab-separated table with two columns:

cell types	gene
guard cell	AT5G26000
mesophyll	AT5G38420

R automatically reads the header `cell types` as `cell.types`.

```
output_dir <- dir_05   # save to 05_annotation/

# Read the bibliography marker table
marcadores <- read.table(file.path(base_dir, "biblio_marks.txt"),
                         header = TRUE, sep = "\t", quote = "")

# Dotplot across numbered Seurat clusters - use this to guide annotation below
hacer_dotplot_marcadores(
  pbmc_harmony, marcadores,
  annot_col = "seurat_clusters",
  outfile   = file.path(output_dir, "dotplot_marcadores_preannotation.pdf"),
  width = 20, height = 10
)
```

12.2 Literature-based annotation

Differentially expressed genes per cluster are identified with a Wilcoxon test and cross-referenced against the same marker table to assign initial labels automatically.

```
markers <- find_markers(pbmc_harmony,
                        output_file = file.path(output_dir, "FindAllMarkers.tsv"))

pbmc_harmony <- annotate_by_markers(pbmc_harmony, markers,
                    reference_file = file.path(base_dir, "biblio_marks.txt"))
```

12.3 Reference-based annotation

Cell-type labels are transferred from a published reference Seurat object for an independent, atlas-validated annotation.

```
esp <- readRDS(file.path(base_dir, "GSE273033_seuratObj_for_publication.rds"))
```

```r
pbmc_harmony <- annotate_by_reference(pbmc_harmony,
                                      reference_obj = esp,
                                      reference_col = "annotation")
# Result stored in: pbmc_harmony$celltype_reference
```

12.4 Annotated clustree

The resolution sweep is repeated overlaying cell-type labels to confirm that the chosen resolution separates biologically distinct populations.

```r
Mode <- function(x) { ux <- unique(x); ux[which.max(tabulate(match(x, ux)))] }

clu <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = dims_use, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = dims_use,
                k.param = k_param, verbose = FALSE)

for (res in resolutions_test)
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)

save_pdf(
  clustree(clu, prefix = "RNA_snn_res.",
           node_label = "celltype_reference", node_label_aggr = "Mode"),
  "clustree_annotated.pdf", w = 14, h = 14
)
```

---

13 Step 11: Gene Expression Visualisation

Individual genes or gene sets can be inspected with violin plots and feature plots, globally or restricted to a single cell type.

```
output_dir <- dir_06   # save to 06_expression/

pbmc_harmony     <- JoinLayers(pbmc_harmony)   # required in Seurat 5 after merge
Idents(pbmc_harmony) <- "celltype_reference"

# Set the gene(s) and cell type of interest
gene              <- "AT5G26000"
genes_of_interest <- c("AT5G26000", "AT5G54250")
celltype          <- "Guard Cell"   # must match exactly a label in celltype_reference
sub_obj           <- subset(pbmc_harmony, idents = celltype)

# Single gene - all cell types
save_vln(VlnPlot(pbmc_harmony, features = gene),    "vln_gene_all.pdf")
save_pdf(FeaturePlot(pbmc_harmony, features = gene), "feature_gene_all.pdf")

# Gene set - all cell types
save_vln(VlnPlot(pbmc_harmony, features = genes_of_interest),
         "vln_geneset_all.pdf", n = length(genes_of_interest))

# Single gene - one cell type
save_vln(VlnPlot(sub_obj, features = gene),    "vln_gene_celltype.pdf")
save_pdf(FeaturePlot(sub_obj, features = gene), "feature_gene_celltype.pdf")
```

---

14 Step 12: Cell-Type Grouping and Manual Curation

14.1 Grouping fine annotations

Fine-grained labels are collapsed into broader biological categories via a named character vector.

```
output_dir <- dir_07   # save to 07_curation/

# Edit this map to match your cell types:
#   Left  : original label (must match exactly)
#   Right : broader category to assign
grouping <- c(
  "Companion Cell"    = "Vascular Cell",
  "Cambium"           = "Vascular Cell",
  "Phloem Parenchyma" = "Vascular Cell",
  "Xylem"             = "Vascular Cell",
  "Sieve Element"     = "Vascular Cell",
  "Meristemoid"       = "Stomatal Line"
)

pbmc_harmony$annotation_agrupada <- recode(pbmc_harmony$celltype_reference,
                                           !!!grouping)

save_pdf(DimPlot(pbmc_harmony, group.by = "annotation_agrupada",
                 label = TRUE, repel = TRUE, raster = FALSE),
         "umap_annotated.pdf")
```

14.2 Manual subclustering and curation

Heterogeneous populations are subclustered and reassigned via an explicit mapping table. **This step must be run interactively.**

The four steps are:

1. **Subcluster** — re-run PCA/UMAP/clustering for each heterogeneous cell type independently.
2. **Inspect** — a single large composite PDF (`07_curation/subclustering_inspection.pdf`) is generated containing all DimPlots and marker FeaturePlots. Open it, decide on the correct identity for each subcluster, then continue to Step 3.
3. **Reassign** — fill in the `reassign` mapping table.
4. **Apply** — corrections are written back to the global object.

```
output_dir <- dir_07   # save to 07_curation/
Idents(pbmc_harmony) <- "annotation_agrupada"

# Step 1. Subcluster heterogeneous types
meristemoid_umap   <- subclustar_tipo(pbmc_harmony, "Stomatal Line")
pavement_cell_umap <- subclustar_tipo(pbmc_harmony, "Pavement Cell")

# Step 2. Generate composite inspection figure and save to disk
# ----------------------------------------------------------------
```

```r
p_meris_dim <- DimPlot(meristemoid_umap, group.by = "cluster_subtipo",
                       label = TRUE, raster = FALSE) +
  ggtitle("Stomatal Line - subclusters")

p_pave_dim  <- DimPlot(pavement_cell_umap, group.by = "cluster_subtipo",
                       label = TRUE, raster = FALSE) +
  ggtitle("Pavement Cell - subclusters")

marker_plots <- lapply(seq_len(nrow(marcadores)), function(i) {
  FeaturePlot(pavement_cell_umap, features = marcadores$gene[i]) +
    ggtitle(paste0(marcadores$cell.types[i], "\n", marcadores$gene[i])) +
    theme(plot.title = element_text(size = 8))
})

ncol_markers <- min(5L, length(marker_plots))
composite_inspect <- (p_meris_dim | p_pave_dim) /
  wrap_plots(marker_plots, ncol = ncol_markers)

ggsave(
  file.path(output_dir, "subclustering_inspection.pdf"),
  composite_inspect,
  width = max(20, ncol_markers * 4),
  height = 10 + ceiling(length(marker_plots) / ncol_markers) * 4,
  limitsize = FALSE
)
# ↑ Open this file before continuing to Step 3 ↑

# Step 3. Reassignment table - fill in after inspecting the PDF
reassign <- list(
  meristemoid_umap = c(
    "0" = "Stomatal Line", "1" = "Stomatal Line",
    "2" = "Pavement Cell", "3" = "Stomatal Line", "4" = "Stomatal Line"
  ),
  pavement_cell_umap = c(
    "0" = "Pavement Cell", "1" = "Pavement Cell",
    "2" = "Pavement Cell", "3" = "Mesophyll",    "4" = "Pavement Cell"
  )
)

# Step 4. Apply corrections
pbmc_harmony$celltype_reference_curated <- pbmc_harmony$annotation_agrupada

for (obj_name in names(reassign)) {
  obj <- get(obj_name)
  pbmc_harmony$celltype_reference_curated[colnames(obj)] <-
    reassign[[obj_name]][obj$cluster_subtipo]
}
```

```
save_pdf(DimPlot(pbmc_harmony, group.by = "celltype_reference_curated",
                 label = TRUE, repel = TRUE, raster = FALSE),
         "umap_curada.pdf")
```

---

15 Step 13: Export to Python (h5ad)

The curated Seurat object is exported to AnnData format for downstream Python analysis (Scanpy, scFates, Palantir — all pre-installed in the Docker image).

```
output_dir <- dir_08   # save to 08_export/

# Export the full curated object
exportar_para_scanpy(pbmc_harmony,
                     file.path(output_dir, "pbmc_harmony_curated.h5ad"))

# To export a single cell type:
# exportar_para_scanpy(
#   subset(pbmc_harmony, subset = celltype_reference_curated == "Guard Cell"),
#   file.path(output_dir, "GuardCell.h5ad")
# )
```

CHAPTER 2 — Pseudobulk Differential Expression and GO Enrichment

16 Step 14: Global Pseudobulk Replicate Correlation

All cells per sample are aggregated into a pseudobulk count vector and Pearson correlations between samples are computed. Replicates from the same condition should cluster together with $r > 0.95$.

```
output_dir <- dir_09   # save to 09_pseudobulk/

pseudobulk <- generate_pseudobulk(pbmc_harmony, group_by = "orig.ident")

save_pdf(plot_replicate_correlation(pseudobulk$by_sample),
         "pseudobulk_correlation.pdf", w = 8, h = 8)
```

---

17 Step 15: Pseudobulk per Cell Type and DESeq2 Differential Expression

17.1 Pseudobulk aggregation

Cells are split by curated cell type, pseudo-replicates are assigned, and counts are summed per replicate.

```
output_dir <- dir_09   # save to 09_pseudobulk/

celular_subsets <- setNames(
  lapply(unique(pbmc_harmony$celltype_reference_curated),
         function(t) subset(pbmc_harmony,
                            subset = celltype_reference_curated == t)),
  gsub("[^[:alnum:]_]", "_", unique(pbmc_harmony$celltype_reference_curated))
)

celular_subsets_replicados <- Filter(Negate(is.null),
                                     lapply(celular_subsets,
                                            asignar_pseudoreplicados))
pseudobulk_list <- lapply(celular_subsets_replicados, hacer_pseudobulk)

dir.create(file.path(output_dir, "pseudobulk_replicas"), recursive = TRUE,
           showWarnings = FALSE)
for (tipo in names(pseudobulk_list))
  write.csv(pseudobulk_list[[tipo]],
            file.path(output_dir, "pseudobulk_replicas",
                      paste0("Pseudobulk_", tipo, ".csv")),
            row.names = TRUE)
```

17.2 DESeq2 analysis

All pairwise condition comparisons are run automatically. Edit the `comparaciones` list to match your condition names.

```
output_dir <- dir_10   # save to 10_deseq2/

# Define your comparisons:
#   conds : c("reference_condition", "treatment_condition")
#   tag   : short label for output file names
comparaciones <- list(
  list(conds = c("0.5N", "5N"), tag = "05_5"),
  list(conds = c("0N",   "5N"), tag = "0_5"),
  list(conds = c("0N", "0.5N"), tag = "0_05")
)

for (tag in sapply(comparaciones, `[[`, "tag"))
  dir.create(file.path(output_dir, tag), recursive = TRUE,
             showWarnings = FALSE)
```

```r
for (tipo in names(pseudobulk_list))
  correr_deseq2(as.matrix(pseudobulk_list[[tipo]]), comparaciones,
                output_dir = output_dir, tipo = tipo)
```

17.3 Volcano plots and differential heatmaps

```r
output_dir <- dir_11   # volcano plots → 11_volcano/

for (comp in comparaciones) {
  tag       <- comp$tag
  csv_files <- list.files(file.path(dir_10, tag),
                          pattern = "\\.csv$", full.names = TRUE)
  if (!length(csv_files)) next

  pdf(file.path(output_dir, paste0("VolcanoPlots_", tag, ".pdf")),
      width = 12, height = 6)
  plots <- list()
  for (f in csv_files) {
    plots <- c(plots, list(hacer_volcano(f)))
    if (length(plots) == 2) { grid.arrange(grobs = plots, ncol = 2); plots <- list() }
  }
  if (length(plots)) grid.arrange(grobs = plots, ncol = 1)
  dev.off()
}

output_dir <- dir_12   # heatmaps → 12_heatmaps/

for (comp in comparaciones) {
  tag      <- comp$tag
  diff_dir <- file.path(output_dir, tag)
  dir.create(diff_dir, recursive = TRUE, showWarnings = FALSE)

  listas      <- lapply(list.files(file.path(dir_10, tag),
                                   pattern = "\\.csv$", full.names = TRUE),
                        procesar_deseq2_resultado, output_dir = diff_dir)
  tabla_logfc <- Reduce(function(x, y) full_join(x, y, by = "gene_id"),
                        lapply(listas, `[[`, "logfc"))
  matriz      <- as.matrix(column_to_rownames(tabla_logfc, "gene_id"))
  matriz[is.na(matriz)] <- 0

  if (nrow(matriz) > 1) {
    pdf(file.path(diff_dir, paste0("heatmap_", tag, ".pdf")),
        width = 14, height = 18)
    tryCatch(hacer_heatmap(matriz),
             error = function(e) message("Heatmap error: ", e$message))
    dev.off()
```

```
  }
}
```

---

18 Step 16: GO Enrichment Analysis

GO enrichment is performed per comparison using `clusterProfiler`. Both full and simplified term sets are tested and visualised as balloon plots.

```
output_dir <- dir_13   # save to 13_go/

# Change for your organism:
#   Arabidopsis thaliana : orgdb = org.At.tair.db, keytype = "TAIR"
#   Homo sapiens         : orgdb = org.Hs.eg.db,   keytype = "ENSEMBL"
#   Mus musculus         : orgdb = org.Mm.eg.db,   keytype = "ENSEMBL"
orgdb      <- org.At.tair.db
keytype    <- "TAIR"
espacio    <- "BP"    # "BP" | "MF" | "CC"
qval       <- 0.05
nivel_poda <- 6       # Maximum GO hierarchy depth for term pruning

universo <- keys(orgdb, keytype = keytype)

for (comp in comparaciones) {
  tag        <- comp$tag
  enr_dir    <- file.path(output_dir, tag)
  tabla_path <- file.path(dir_12, tag, "tabla_diferenciales.tsv")
  dir.create(enr_dir, recursive = TRUE, showWarnings = FALSE)
  if (!file.exists(tabla_path)) next

  tabla     <- read.table(tabla_path, header = TRUE, row.names = 1, sep = "\t")
  go_total  <- correr_enriquecimiento_go(tabla, universo, espacio,
                 orgdb = orgdb, keytype = keytype, simplificar = FALSE,
                 output_dir = enr_dir)
  go_simple <- correr_enriquecimiento_go(tabla, universo, espacio,
                 orgdb = orgdb, keytype = keytype, simplificar = TRUE,
                 output_dir = enr_dir)

  go_total_podado  <- podar_go(go_total,  nivel_poda, espacio, qval,
                               simplificar = FALSE, output_dir = enr_dir)
  go_simple_podado <- podar_go(go_simple, nivel_poda, espacio, qval,
                               simplificar = TRUE,  output_dir = enr_dir)

  pdf(file.path(enr_dir, paste0("GO_enrichment_", tag, ".pdf")),
      width = 18, height = 18)
  tryCatch({
    print(graficar_go_balones(go_total))
    print(graficar_go_balones(go_simple))
    print(graficar_go_balones(go_total_podado))
    print(graficar_go_balones(go_simple_podado))
  }, error = function(e) message("GO plot error: ", e$message))
  dev.off()
```

```
}
```

---

19 Adapting the Pipeline to Other Organisms

Three parameters control the organism-specific parts of the analysis. Each is defined immediately before the section that uses it in `scrnaseq_pipeline.R`.

Mitochondrial / chloroplast gene patterns (QC section):

Organism	`mt_pattern`	`cp_pattern`
Arabidopsis thaliana	`"^ATMG"`	`"^ATCG"`
Human	`"^MT-"`	`NULL`
Mouse	`"^mt-"`	`NULL`

GO enrichment database (GO enrichment section):

```
# Arabidopsis
orgdb <- org.At.tair.db;  keytype <- "TAIR"

# Human
orgdb <- org.Hs.eg.db;    keytype <- "ENSEMBL"

# Mouse
orgdb <- org.Mm.eg.db;    keytype <- "ENSEMBL"
```

DESeq2 comparisons — edit the `comparaciones` list to match your condition names:

```
comparaciones <- list(
  list(conds = c("Control", "Treatment"), tag = "ctrl_vs_treat"),
  list(conds = c("Control", "HighDose"),  tag = "ctrl_vs_high")
)
```

20 Output File Summary

Folder	File	Description
00_workflow/	pipeline_workflow.pdf	Visual pipeline overview
01_qc/	qc_prefilter.pdf	QC violin plots before filtering
02_filtering/	qc_postfilter.pdf	QC violin plots after filtering
03_integration/	umap_preharmony.pdf	UMAP before batch correction
04_clustering/	elbow_plot.pdf	K-means WSS elbow plot
04_clustering/	clustree.pdf	Resolution sweep visualisation
04_clustering/	umap_postharmony.pdf	UMAP after Harmony, by sample
04_clustering/	umap_seuratclusters.pdf	UMAP coloured by cluster
04_clustering/	cell_count_per_sample.pdf	Total cells per sample
05_annotation/	dotplot_marcadores_preannotation.pdf	Pre-annotation marker dot plot
05_annotation/	FindAllMarkers.tsv	Cluster marker gene table
05_annotation/	clustree_annotated.pdf	Resolution sweep with cell-type labels
06_expression/	vln_*.pdf, feature_*.pdf	Violin and feature plots
07_curation/	umap_annotated.pdf	UMAP with grouped cell-type labels
07_curation/	subclustering_inspection.pdf	Composite subclustering figure
07_curation/	umap_curada.pdf	UMAP after manual curation
08_export/	pbmc_harmony_curated.h5ad	Exported AnnData object
09_pseudobulk/	pseudobulk_correlation.pdf	Replicate Pearson correlation heatmap
09_pseudobulk/	pseudobulk_replicas/*.csv	Pseudobulk count matrices per cell type
10_deseq2/	<tag>/*.csv	DESeq2 results per cell type
11_volcano/	VolcanoPlots_*.pdf	Volcano plots per comparison
12_heatmaps/	<tag>/heatmap_*.pdf	Log2FC heatmaps
13_go/	<tag>/GO_enrichment_*.pdf	GO enrichment balloon plots